

```

/* * ===== * Mini Linux
Shell — CS 325 Operating Systems Lab Project * Author : Ali * File : minishell.c (or test.c) * Build :
gcc -o minishell minishell.c *
===== * Features
implemented * _____ * 1. Execute external commands (fork + execvp) *
2. Background process execution (command &) * 3. Pipes between two commands (cmd1 | cmd2)
* 4. Command history (last 100 cmds) (history built-in) * 5. Custom built-in commands (cd, help,
history, * clear, pwd, exit, ask) * * Concepts demonstrated * _____ *
fork() — create child process * • exec() — replace process image with a new program * • pipe() —
unidirectional inter-process communication * • wait() — parent waits for child termination * • getpid()
— retrieve process IDs * • chdir() — system call for cd * • system() — IPC via external API calling
(Local LLM) */ #include #include #include #include #include #include #include #include #include #include /* _____
constants _____ */ #define
MAX_INPUT 1024 #define MAX_ARGS 64 #define MAX_HISTORY 100 #define SHELL_NAME
"minishell" /* _____ colour helpers (ANSI escape codes) _____ */ #define
RESET "\033[0m" #define BOLD "\033[1m" #define RED "\033[1;31m" #define GREEN
"\033[1;32m" #define YELLOW "\033[1;33m" #define CYAN "\033[1;36m" #define BLUE
"\033[1;34m" #define MAGENTA "\033[1;35m" /* _____ history storage
_____ */ typedef struct { char
cmd[MAX_INPUT]; time_t timestamp; } HistoryEntry; static HistoryEntry history[MAX_HISTORY];
static int history_count = 0; /*
=====
UTILITY FUNCTIONS *
===== */ /*
Save a command to the in-memory history ring */ void history_add(const char *line) { if (strlen(line)
== 0) return; /* avoid duplicate consecutive entries */ if (history_count > 0 &&
strcmp(history[history_count - 1].cmd, line) == 0) return; int idx = history_count % MAX_HISTORY;
strncpy(history[idx].cmd, line, MAX_INPUT - 1); history[idx].timestamp = time(NULL);
history_count++; } /* Print the history list */ void history_print(void) { int total = (history_count <
MAX_HISTORY) ? history_count : MAX_HISTORY; int start = (history_count > MAX_HISTORY) ?
history_count - MAX_HISTORY : 0; printf(CYAN "\n — Command History
\n\n" RESET); for (int i = 0; i < total; i++) { int idx = (start + i) %
MAX_HISTORY; int num = start + i + 1; char tbuf[32]; struct tm *tm_info =
localtime(&history[idx].timestamp); strftime(tbuf, sizeof(tbuf), "%H:%M:%S", tm_info);
printf(YELLOW " %3d" RESET " [%s] %s\n", num, tbuf, history[idx].cmd); } printf(CYAN "
\n\n" RESET); } /* Tokenise a line
into argv[], return argc */ int parse_args(char *line, char **argv) { int argc = 0; char *token =
strtok(line, " \t\n"); while (token && argc < MAX_ARGS - 1) { argv[argc++] = token; token =
strtok(NULL, " \t\n"); } argv[argc] = NULL; return argc; } /* Print the prompt */ void
print_prompt(void) { char cwd[512]; if (getcwd(cwd, sizeof(cwd)) == NULL) strcpy(cwd, "?"); /*
shorten home dir to ~ */ char *home = getenv("HOME"); if (home && strncmp(cwd, home,
strlen(home)) == 0) { char short_cwd[512]; snprintf(short_cwd, sizeof(short_cwd), "~%s", cwd +
strlen(home)); printf(GREEN BOLD "%s" RESET ":" BLUE "%s" RESET "$ ", SHELL_NAME,
short_cwd); } else { printf(GREEN BOLD "%s" RESET ":" BLUE "%s" RESET "$ ", SHELL_NAME,
cwd); } fflush(stdout); } /* Print welcome banner */ void print_banner(void) { printf("\n"); printf(CYAN "
\n\n" RESET); printf(CYAN " || "
BOLD YELLOW "Mini Linux Shell — OS Lab Project" RESET CYAN " || \n\n"); printf(CYAN " || "
RESET "CS 325 · Air University Kharian" CYAN " || \n\n"); printf(CYAN " || " RESET "Type " GREEN
"help" RESET " for built-in commands" CYAN " || \n\n"); printf(CYAN "
|| \n\n" RESET); printf("\n"); } /*
Print help */ void print_help(void) { printf(CYAN "\n — Built-in Commands
\n\n" RESET); printf(GREEN " %-18s" RESET
"%s\n", "cd
", "Change directory"); printf(GREEN " %-18s" RESET "%s\n", "pwd", "Print working
directory"); printf(GREEN " %-18s" RESET "%s\n", "history", "Show command history");
printf(GREEN " %-18s" RESET "%s\n", "clear", "Clear the terminal screen"); printf(GREEN "
%-18s" RESET "%s\n", "ask \"...\"", "Ask the local AI a question"); printf(GREEN " %-18s"

```

```

RESET "%s\n", "help", "Show this help message"); printf(GREEN " %-18s" RESET "%s\n",
"exit", "Exit the shell"); printf(CYAN "\n — Usage Examples
                                     \n" RESET); printf(YELLOW " %-30s"
RESET "%s\n", "ls -la", "Normal command"); printf(YELLOW " %-30s" RESET "%s\n", "sleep
10 &", "Background process"); printf(YELLOW " %-30s" RESET "%s\n", "ls -la | grep .c",
"Pipe"); printf(YELLOW " %-30s" RESET "%s\n", "ask \"explain fork()\"", "Talk to AI");
printf(CYAN "
                                     \n\n"
RESET); } /*

```

* CORE EXECUTION FUNCTIONS *

```

/* /* * execute_simple — fork a child and exec a single command. * If background == 1 the
parent does NOT wait. */ void execute_simple(char **argv, int background) { pid_t pid =
fork(); /* ← SYSTEM CALL: fork */ if (pid < 0) { perror(RED "fork failed" RESET); return; } if
(pid == 0) { /* — CHILD PROCESS — */ execvp(argv[0], argv); /* ← SYSTEM CALL: exec
*/ /* execvp only returns on error */ fprintf(stderr, RED "%s: command not found\n" RESET,
argv[0]); exit(EXIT_FAILURE); } /* — PARENT PROCESS — */ if (background) {
printf(MAGENTA "[background] PID %d started: %s\n" RESET, pid, argv[0]); /* SIGCHLD will
clean up or we reap lazily in the next prompt */ } else { int status; waitpid(pid, &status, 0); /* ←
SYSTEM CALL: wait */ if (WIFEXITED(status) && WEXITSTATUS(status) != 0)
printf(YELLOW "[exit status: %d]\n" RESET, WEXITSTATUS(status)); } } /* * execute_pipe —
run cmd1 | cmd2 using a kernel pipe. */ void execute_pipe(char **argv1, char **argv2) { int
fd[2]; if (pipe(fd) == -1) { /* ← SYSTEM CALL: pipe */ perror(RED "pipe failed" RESET);
return; } /* — Left side of pipe (writer) — */ pid_t pid1 = fork(); /* ← SYSTEM CALL: fork */ if
(pid1 < 0) { perror(RED "fork" RESET); return; } if (pid1 == 0) { close(fd[0]); /* not reading */
dup2(fd[1], STDOUT_FILENO); /* stdout → pipe write-end */ close(fd[1]); execvp(argv1[0],
argv1); /* ← SYSTEM CALL: exec */ fprintf(stderr, RED "%s: command not found\n" RESET,
argv1[0]); exit(EXIT_FAILURE); } /* — Right side of pipe (reader) — */ pid_t pid2 = fork(); /*
← SYSTEM CALL: fork */ if (pid2 < 0) { perror(RED "fork" RESET); return; } if (pid2 == 0) {
close(fd[1]); /* not writing */ dup2(fd[0], STDIN_FILENO); /* stdin ← pipe read-end */
close(fd[0]); execvp(argv2[0], argv2); /* ← SYSTEM CALL: exec */ fprintf(stderr, RED "%s:
command not found\n" RESET, argv2[0]); exit(EXIT_FAILURE); } /* parent closes both ends
and waits for both children */ close(fd[0]); close(fd[1]); waitpid(pid1, NULL, 0); /* ← SYSTEM
CALL: wait */ waitpid(pid2, NULL, 0); } /* * handle_builtin — check and execute built-in
commands. * Returns 1 if the command was a built-in, 0 otherwise. */ int handle_builtin(char
**argv, int argc) { if (argc == 0) return 1; /* exit */ if (strcmp(argv[0], "exit") == 0) {
printf(GREEN "Goodbye!\n" RESET); exit(EXIT_SUCCESS); } /* help */ if (strcmp(argv[0],
"help") == 0) { print_help(); return 1; } /* history */ if (strcmp(argv[0], "history") == 0) {
history_print(); return 1; } /* pwd */ if (strcmp(argv[0], "pwd") == 0) { char cwd[512]; if
(getcwd(cwd, sizeof(cwd))) /* ← SYSTEM CALL: getcwd */ printf("%s\n", cwd); else
perror("pwd"); return 1; } /* clear */ if (strcmp(argv[0], "clear") == 0) { printf("\033[2J\033[H"); /*
ANSI: clear screen + move cursor home */ return 1; } /* cd */ if (strcmp(argv[0], "cd") == 0) {
const char *dir = (argc > 1) ? argv[1] : getenv("HOME"); if (!dir) dir = "/"; if (chdir(dir) != 0) /* ←
SYSTEM CALL: chdir */ fprintf(stderr, RED "cd: %s: %s\n" RESET, dir, strerror(errno)); return
1; } /* — AI INTEGRATION — */ if (strcmp(argv[0], "ask") == 0) { if (argc < 2) {
printf(RED "Usage: ask \"your question here\"\n" RESET); return 1; } // Declare the variables
before using them to satisfy the compiler char prompt[MAX_INPUT] = ""; char
final_cmd[MAX_INPUT + 512]; // Combine arguments into a single string for (int i = 1; i <
argc; i++) { strncat(prompt, argv[i], MAX_INPUT - strlen(prompt) - 1); strncat(prompt, " ",
MAX_INPUT - strlen(prompt) - 1); } // Construct the CURL payload for tinyllama
snprintf(final_cmd, sizeof(final_cmd), "curl -s -X POST http://localhost:11434/api/generate " "-
d '{\"model\": \"tinyllama\", \"prompt\": \"%s\", \"stream\": false}' " "| jq -j '.response'", prompt);
printf(MAGENTA "\nAI is thinking...\n" RESET); // Execute the API call system(final_cmd);
printf("\n\n"); return 1; } return 0; /* not a built-in */ } /*

```

* COMMAND DISPATCHER *

```

/* void dispatch(char *input) { /* strip trailing newline */ input[strcspn(input, "\n")] = '\0'; /* skip
blank lines */ if (strlen(input) == 0) return; history_add(input); /* — check for background flag
— */ int background = 0; size_t len = strlen(input); if (len > 0 && input[len - 1] == '&') {
background = 1; input[len - 1] = '\0'; /* trim trailing space before & */ while (len > 1 &&
input[len - 2] == ' ') input[--len - 1] = '\0'; } /* — check for pipe — */ char *pipe_pos =
strchr(input, '|'); if (pipe_pos) { /* split into two halves */ *pipe_pos = '\0'; char *left = input;
char *right = pipe_pos + 1; char *argv1[MAX_ARGS], *argv2[MAX_ARGS]; /* tokenise each
half on a copy so strtok doesn't clobber */ char left_copy[MAX_INPUT],
right_copy[MAX_INPUT]; strncpy(left_copy, left, MAX_INPUT - 1); strncpy(right_copy, right,
MAX_INPUT - 1); parse_args(left_copy, argv1); parse_args(right_copy, argv2); if (argv1[0] ==
NULL || argv2[0] == NULL) { fprintf(stderr, RED "syntax error: invalid pipe\n" RESET); return;
} execute_pipe(argv1, argv2); return; } /* — single command — */ char copy[MAX_INPUT];
strncpy(copy, input, MAX_INPUT - 1); char *argv[MAX_ARGS]; int argc = parse_args(copy,
argv); if (argc == 0) return; /* try built-ins first */ if (!handle_builtin(argv, argc))
execute_simple(argv, background); } /*

```

* MAIN — shell read-eval-print loop *

```

/* int main(void) { char input[MAX_INPUT]; print_banner(); while (1) { /* reap any finished
background children without blocking */ while (waitpid(-1, NULL, WNOHANG) > 0) ;
print_prompt(); if (fgets(input, sizeof(input), stdin) == NULL) { /* EOF (Ctrl-D) */ printf(GREEN
"\nGoodbye!\n" RESET); break; } dispatch(input); } return 0; }

```